

API

Version 1.0

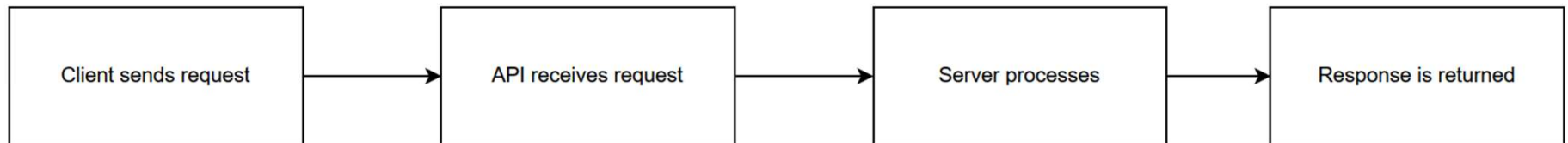
Author: Jagat Jeevan

Date: 29-04-2026

What is an API

- An API is an interface that enables different software applications to communicate with each other.
- Technical writers are responsible for creating API documentation.
- APIs define the rules for how requests and responses are formatted between systems.

API Flow



API Structure

https://api.example.com/users

An API URL consists of the following components

- Protocol : **https://**
- Base URL : **api.example.com**
- Endpoint : **/users**

Why we use APIs

- **Integration** - connects separate applications to work as a unified system
- **Efficiency** - reduces development time by reusing existing APIs
- **Security** - exposes only authorized data, keeping internal systems protected

Integration

Connect subsystems or applications so they work together as a single, cohesive unit.

Example : Amazon  PhonePe  Amazon

When you purchase an item on Amazon, the Amazon application uses an API to redirect you securely to PhonePe for payment processing. After the transaction is completed successfully, you are returned to the Amazon application.

In the above example, the API enables communication between two separate applications to complete a single task.

Efficiency

Developers can integrate existing APIs to build applications faster and reduce errors. Rather than building from scratch, developers can integrate existing APIs directly into their applications.

Example – A food delivery app can integrate the Google Maps API for navigation instead of building its own mapping system.

Security

APIs improve security by ensuring that only the specific data that is requested and authorized is shared. The rest of the system's internal data remains protected.

Example - A banking app that exposes only your account balance to a third-party budgeting app, without sharing your full transaction history or credentials.

API Architectures

- REST (Representational State Transfer)
- SOAP (Simple Object Access Protocol)
- GraphQL
- gRPC
- WebSockets

REST (Representational State Transfer)

- Most widely used
- Uses GET, POST, PUT, DELETE methods
- Data format – JSON
- Used in Web apps, mobile apps

SOAP (Simple Object Access Protocol)

- Data format – XML
- Built-in security and error handling
- Used in banking, financial services

GraphQL

- Developed by Meta
- Client requests exactly what data it needs

gRPC

- Developed by Google
- Uses Protocol Buffers (faster than JSON)
- Used in microservices, high-performance systems

WebSockets

- Real-time, two-way communication
- Used in chat apps, live notifications, gaming

What is JSON

- JavaScript Object Notation.
- Text-based format used to send data between systems and sub systems
- Highly popular as it is easy for humans to read and write, and easy for machines to parse and generate
- When our APIs are talking to each other like in our **Amazon and PhonePe example**, the systems need a common language to understand the data being passed back and forth. JSON is the industry standard for API communication because it is structured, predictable, and efficient to process.

JSON

```
{  
  "userName": "XYZ",  
  "orderId": 98444,  
  "paymentStatus": "Success",  
  "totalAmount": 2000.00  
}
```

API Writing

1. Authorization/Authentication (verifying identity)
2. Endpoint (to get the specific information)
3. Methods
4. Parameter (specific details added in endpoint to narrow down or data you get back)
5. Request and Response
6. Error handling

API Methods

Method	Purpose
GET	Retrieve data
POST	Create data
PUT	Replace data
PATCH	Update partial data
DELETE	Remove data

CRUD Functions

- **Create** (POST) – Addition of new data to the database
- **Read** (GET) – Fetching the data from the database.
- **Update** (PUT) - Replaces an entire resource.
(PATCH) – Replaces a specific field or criteria.
- **Delete** – Remove data from database.

HTTP Status Codes

- Three-digit numbers returned by a server to indicate the outcome of a client's request.
- They act as a standard method for applications to indicate whether a request was successful, failed, or requires further action.

- **1xx (Informational):** The server received the request and is processing it.
- **2xx (Success):** The request was successfully received and accepted (For example, 200 OK)
- **3xx (Redirection):** The client must take additional action to complete the request.
- **4xx (Client error):** The request contains bad syntax or cannot be fulfilled by the server (For example, 400 Bad Request, 404 Not Found).
- **5xx (Server error):** The server failed to fulfill a valid request (For example, 500 Internal Server Error).